Lehrevaluation WS25/26

Technische Universität Berlin

Fortgeschrittene Sortierverfahren

Manfred Hauswirth | Open Distributed Systems | Einführung in die Programmierung, WS 25/26

Open Distributed Sytems
ODS

Lehrevaluation

- Aktuell finden die Lehrevaluationen für die Lehrveranstaltungen des Wintersemesters 2025/2026 statt.
- Link zu der Umfrage
- Die Teilnahme ist bis zum 23.01.2026 möglich.
- Dauer ca. 5 Minuten

Praktischer Test am 17.01.2026

Technische Universität Berlin

- Im HE101
- Zuweisung des Zeitraumes auf ISIS einsehbar
- Einlass (zirka 15-30 Minuten vor Einlass vor Ort sein und Witterung bei Anfahrt berücksichtigen)
  - Gruppe 1 : 11:00 Uhr
  - Gruppe 2 : 13:00 Uhr
  - Gruppe 3 : 15:00 Uhr
- Studierendenausweis mitführen
- Keine weiteren Hilfsmittel zugelassen
- Weitere Hinweise siehe Ankündigung auf ISIS

Open Distributed Sytems
ODS

Rückblick

Technische Universität Berlin

VL 0 „Organisation und Inhalt“: Ablauf der Vorlesung, Termine
VL 1 „Algorithmen, Pseudocode, Sortieren I“: Insertion Sort
VL 2 „Algorithmen, Pseudocode, Sortieren II“: Selection Sort, Bubble Sort, Count Sort
VL 3 „Laufzeit und Speicherplatz“: Laufzeitanalyse der vorgestellten Sortierverfahren
VL 4 „Einfache Datenstrukturen“: Arrays, verkettete Listen, Structs in C, Stack, Queue
VL 5 „Bäume“: Binärbäume, Baumtraversierung, Laufzeitanalyse Baumoperationen
VL 6 „Teile und Herrsche I“: Einführung der algorithmischen Methode, Merge Sort
VL 7 „Korrektheitsbeweise“: Rechnermodel, Beispielbeweise
VL 8 „Dateien in C“: Dateien, Dateisysteme, Verzeichnisse, Dateiverwaltung mit C
VL 9 „Prioritätenschlangen/Halden/Heaps “: Heap Sort, Binärer Heap, Heap Operationen
VL 10 „Fortgeschrittene Sortierverfahren“: Quick Sort, Radix Sort
VL 11 „AVL Bäume“: Definition, Baumoperationen, Traversierung
VL 12 „Teile und Herrsche II“: Generalisierung des algorithmischen Prinzips, Mastertheorem
VL 13 „Q & A“: Offene Vorlesung/Wiederholung

Open Distributed Sytems
ODS

Überblick

- Sortieren ist ein sehr intensiv untersuchtes Problem
- Es gibt eine große Zahl von Algorithmen mit jeweils verschiedenen Varianten
- Generell ordnet man Sortierverfahren in zwei Gruppen:
  - Vergleichende Sortierverfahren
    - Einfache Sortierverfahren
    - Fortgeschrittene Sortierverfahren
  - Nicht vergleichende Sortierverfahren

Überblick

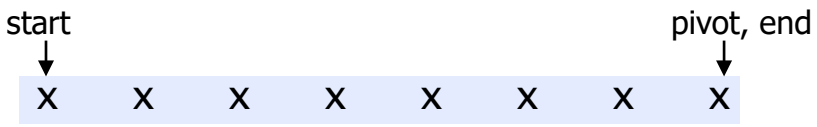
- Einfache vergleichende Sortierverfahren
  - Sortieren durch Einfügen (insertion sort)
  - Sortieren durch Auswählen (selection sort)
  - Sortieren durch Vertauschen (bubble sort)
- Fortgeschrittene vergleichende Sortierverfahren
  - Sortieren durch Mischen (merge sort)
  - Sortieren mittels Heapify (heap sort)
  - Sortieren durch Gruppieren (quick sort)
- Nicht vergleichende Sortierverfahren
  - Sortieren durch Zählen (count sort)
  - Sortieren durch Fachverteilen (radix sort)

Quicksort

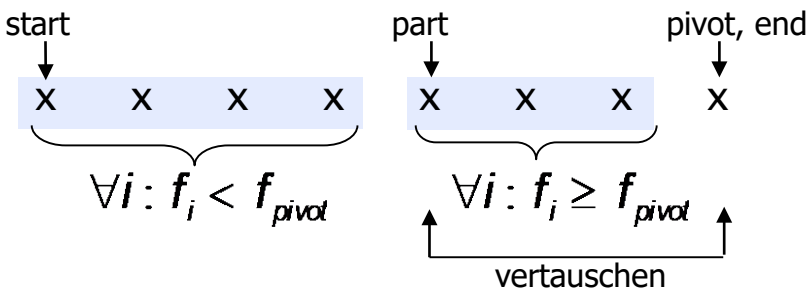
- Sortieren durch Gruppieren
  - Berühmt, schnell, breit einsetzbar
  - Nutzt das Prinzip „Teile und Herrsche“
- Arbeitsweise
  - Die Folge wird in zwei Teilfolgen gruppiert (partition), so dass jedes Element der ersten Folge kleiner ist als jedes Element der zweiten.
  - Auf diese Teilfolgen wird das Prinzip dann rekursiv angewendet.
  - Die Rekursion endet, wenn man bei einelementigen Teilfolgen angelangt ist.
  - Als Trennelement (pivot) für das Gruppieren kann ein beliebiges Folgenelement verwendet werden, z.B. das letzte, das erste, oder das mittlere.

Quicksort: Prinzip

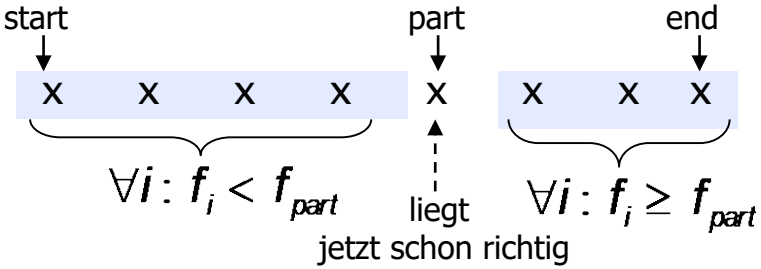
Anfangssituation:



Nach Gruppieren:



Nach Vertauschen:



Komplexität von Quicksort

- Wenn es gelingt, die Teilfolgen durch das Gruppieren jeweils zu halbieren, erhalten wir eine Komplexität von: $T_{bc} = O(n \log n)$

- Wird jeweils nur eine Teilfolge konstanter Länge abgespalten, so wird n mal gruppiert und daher: $T_{wc} = O(n^2)$

- Im Durchschnitt ist allerdings die Länge der Intervalle abhängig von *n* und man braucht nur zweimal mehr Operationen als im besten Fall: $T_{ac} = O(n \log n)$

Quicksort-Varianten

Vorgeschlagene Varianten betreffen die Wahl des Trennwerts für das Gruppieren, der in der vorliegenden Form für das schlechte Verhalten bei nahezu sortierten Folgen verantwortlich ist:

1. Trennindex zufällig aus $[start, end]$
2. Trennwert als Median aus drei Elementen $\left\{f_{start}, f_{\left\lceil \frac{start+end}{2} \right\rceil}, f_{end}\right\}$

- Beide Varianten haben sich gut bewährt und sorgen auch bei fast sortierten Folgen für O(n log n).
- Es ist außerdem vorteilhaft, die kleinere Teilfolge zuerst zu sortieren (Rekursionstiefe minimieren).

Quicksort mit Listen: Gruppieren

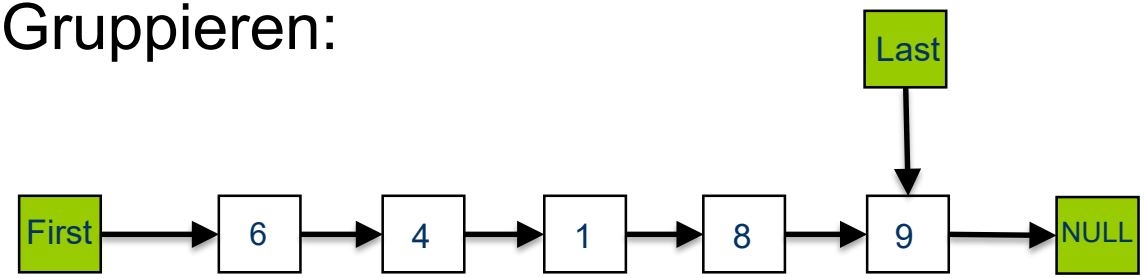
Technische Universität Berlin

- Zerlegen einer Folge in zwei Teilfolgen, wovon die eine die kleineren (<) Elemente, die andere die größeren (≥) Elemente enthält
  - Als Trennwert (pivot) wird oft der Wert des ersten Elements genommen.
  - Statt des Vertauschens (swap) werden die Elemente in zwei neue Listen (left, right) versetzt (move).
  - Diese werden dann rekursiv sortiert und zu der Resultatliste zusammengesetzt.
  - Die Komplexität des Gruppierungsverfahrens ist, wie bei den Arrays, O(n), da die Liste genau einmal abgelaufen wird und jeweils konstanter Aufwand entsteht.

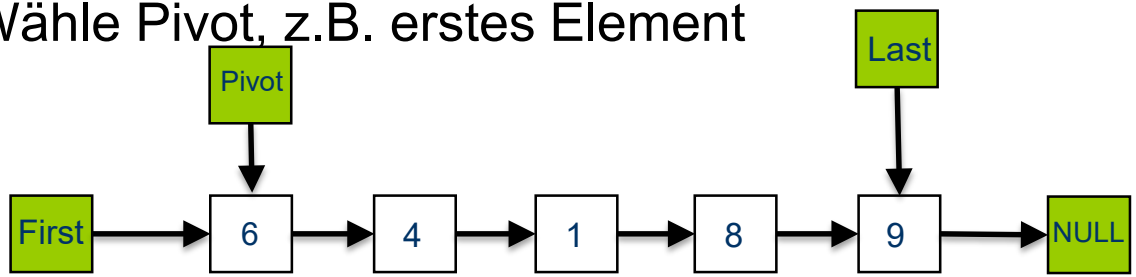
Open Distributed Sytems
ODS

Sortieren durch Gruppieren (Quicksort)

- Vor Gruppieren:



- Partition
  1. Wähle Pivot, z.B. erstes Element



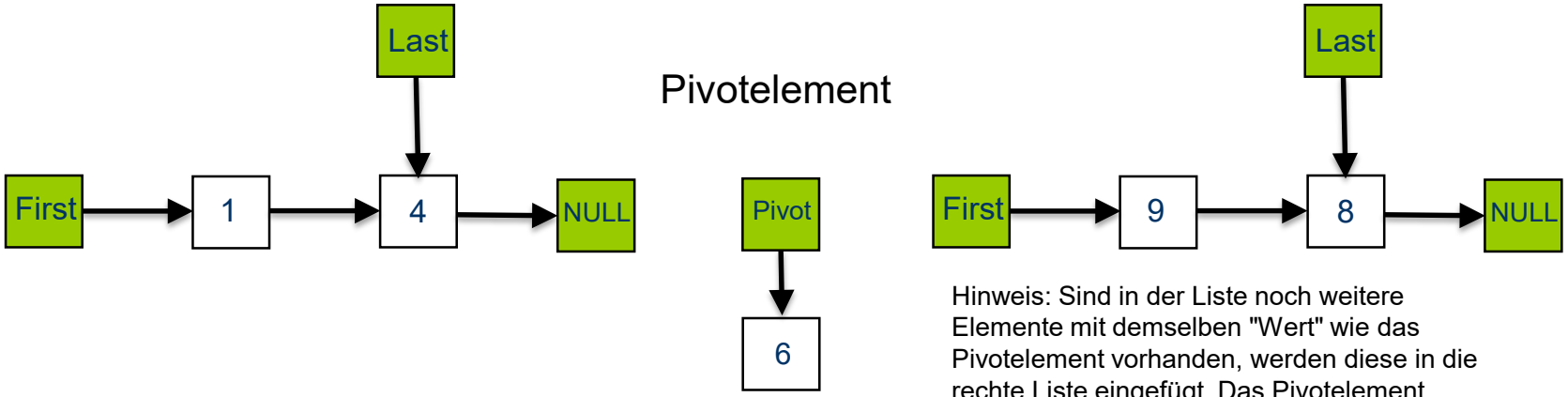
Sortieren durch Gruppieren (Quicksort)

- Partition

  2. Liste aufteilen (mit Einfügen am Anfang)

Neue Liste links mit Elementen < Pivot

Neue Liste rechts mit Elementen ≥ Pivot



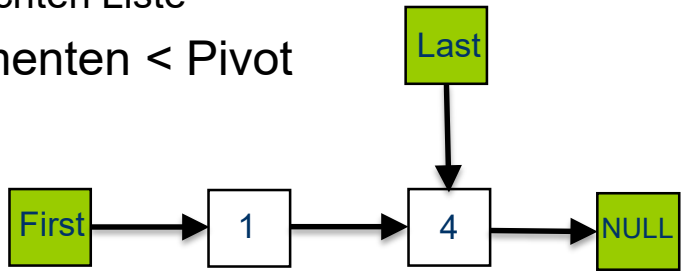
Hinweis: Sind in der Liste noch weitere Elemente mit demselben "Wert" wie das Pivotelement vorhanden, werden diese in die rechte Liste eingefügt. Das Pivotelement selbst ist immer Rückgabewert der Partition.

Stabilität von Quicksort

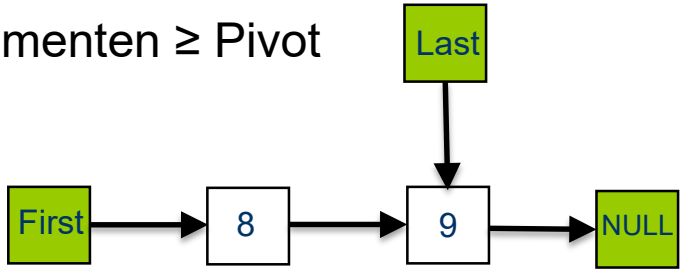
- Erinnerung:
  - Sortierungsalgorithmus stabil ⇔ Elemente mit gleichem Wert in der gleichen Reihenfolge bei der Ausgabe
  - Bsp. – stabiles Sortieren: 2, 3, 2, 5, 2, 1, 8 ⇒ 1, 2, 2, 2, 3, 5, 8
  - Bsp. – instabiles Sortieren: 2, 3, 2, 5, 2, 1, 8 ⇒ 1, 2, 2, 2, 3, 5, 8
- Quicksort ist nur dann stabil, wenn
  - alle Elemente mit dem gleichen Wert des gewählten Pivots, die vor bzw. nach dem Pivot kommen, auch entsprechend in die linke bzw. rechte Liste gehen.
- Streng kleiner (<) links, größer gleich (≥) rechts mit Einfügen am Ende
  - erstes Element als Pivot ⇒ stabil
    - **2**, 3, 2, 5, 2, 1, 8 ⇒ l={1}, **2**, r={3, 2, 5, 2, 8} ⇒ 2 bleibt vor 2 ☺
  - anderes Element als Pivot ⇒ instabil
    - 2, 3, **2**, 5, 2, 1, 8 ⇒ l={1}, **2**, r={2, 3, 5, 2, 8} ⇒ 2 jetzt vor 2 ☹

Sortieren durch Gruppieren (Quicksort)

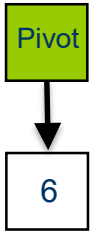
- Quicksort:
  3. Rekursives Sortieren der linken und rechten Liste
     - Sortierte linke Liste mit Elementen < Pivot



     - Sortierte rechte Liste mit Elementen ≥ Pivot

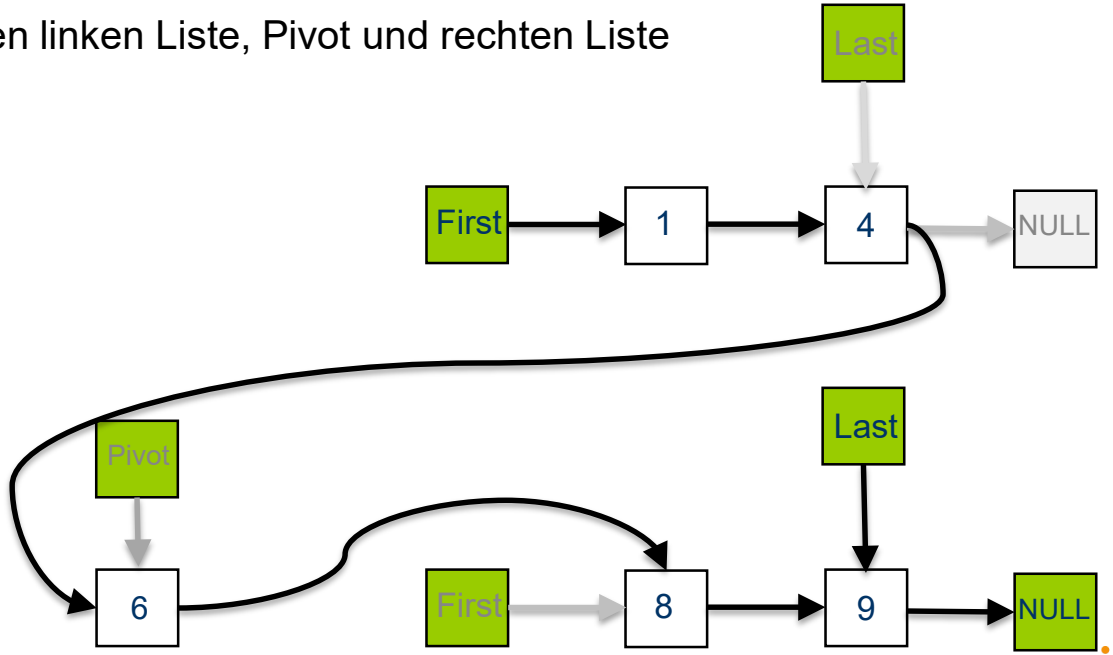


     - Pivotelement



Sortieren durch Gruppieren (Quicksort)

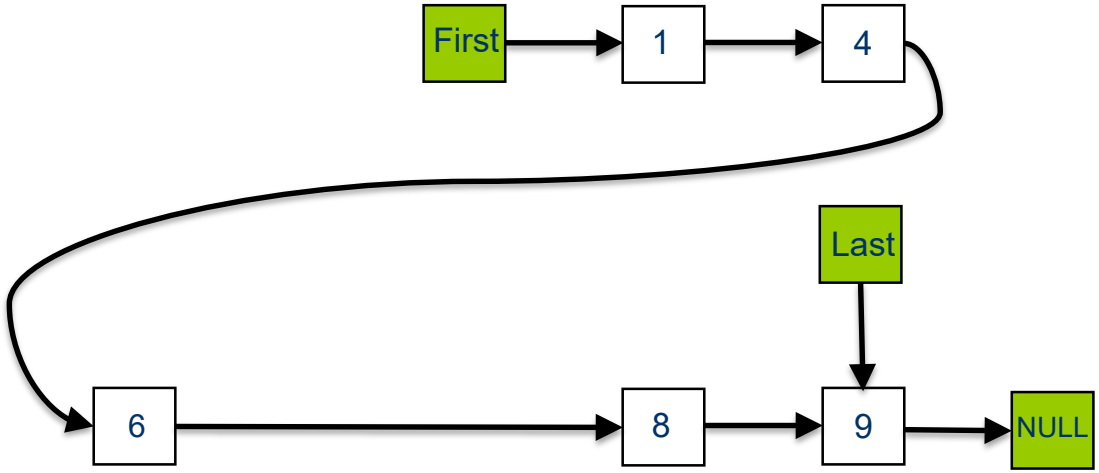
- Quicksort:
  4. Zusammenfügen linken Liste, Pivot und rechten Liste



Sortieren durch Gruppieren (Quicksort)

- Quicksort:

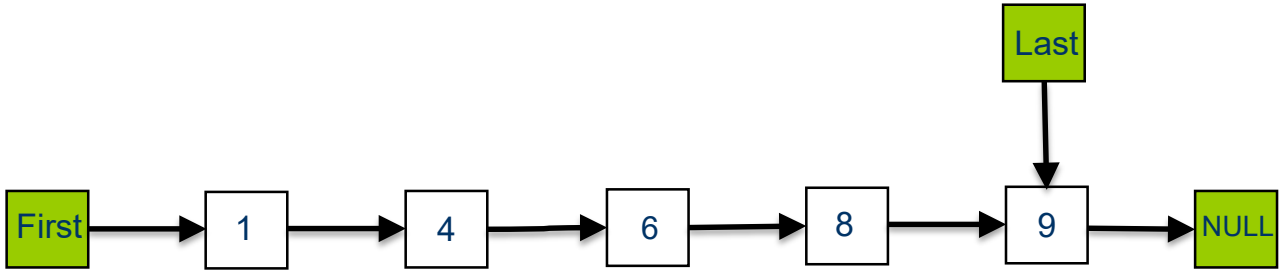
  Resultat: Sortierte Liste



Sortieren durch Gruppieren (Quicksort)

- Quicksort:

  Resultat: Sortierte Liste



Quicksort – Pseudocode

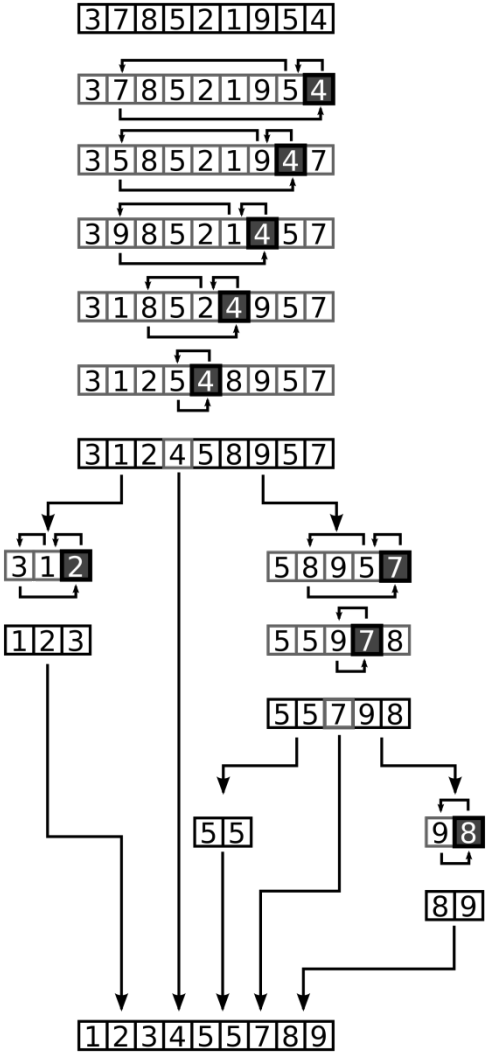
```
QuickSort ( list to sort )
    // Abbruchbedingung
    if ( tosort.first = tosort.last )
        return
    else
        list  left,right

        pivot ← Partition(tosort, left, right)
        // Rekursion
        QuickSort (left)
        QuickSort (right)
    // Listen zusammenfügen
    if ( left.first = NULL )    // Test auf Gleichheit !
            tosort.first ← pivot
    else
            tosort.first ← left.first
            left.last.next ← pivot

    if ( right.first = NULL )  // Test auf Gleichheit !
            pivot.next ← NULL
            tosort.last ← pivot
    else
            pivot.next ← right.first
            tosort.last ← right.last
```

Beispiel

- Quicksort mit Array
- Pivot: letztes Element (dunkler Hintergrund)
- Arrays werden von links nach rechts abgearbeitet
- Statt linker und rechter Liste: Ringtausch



Technische Universität Berlin

Open Distributed Sytems ODS

Quicksort: Zusammenfassung

- Sortieren durch Gruppieren (quick sort)
  - Berühmt, schnell, breit einsetzbar
  - Nutzt das Prinzip „Teile und Herrsche“
- Arbeitsweise
  - Die Folge wird in zwei Teilfolgen gruppiert (partition), so dass jedes Element der ersten Folge kleiner ist als jedes Element der zweiten.
  - Auf diese Teilfolgen wird das Prinzip dann rekursiv angewendet.
  - Die Rekursion endet, wenn man bei einelementigen Teilfolgen angelangt ist.
  - Kann sowohl auf Arrays als auch auf Listen eingesetzt werden
  - Als Trennelement (pivot) für das Gruppieren kann
    - Bei Arrays ein beliebiges Folgenelement verwendet werden, z.B. das letzte, erste oder mittlere Element.
    - Bei Listen am einfachsten das erste oder letzte Element.

Sortieren

- Standard Sortierverfahren (Bubble-sort, Insertion-sort, Selection-sort, Quicksort, Merge-sort) sind nicht nur auf Arrays sondern auch auf verkettete Listen anwendbar.
  - Die Komplexität ist dieselbe.
- Lediglich die auf Indexrechnung beruhenden schnellen Verfahren (count sort) können nicht verwendet werden.

Anmerkungen: Quicksort vs. Merge-Sort

Technische Universität Berlin

- Beide Verfahren, Quicksort und Merge-Sort, benötigen zusätzlichen Speicher
- Merge-Sort, weil das Mischen in linearer Zeit nur mit einem zweiten Array möglich ist.
- Quicksort, weil die rekursiven Aufrufe den Programmstapel (program stack) erheblich anwachsen lassen.
  - Dies kann man mildern, wenn man das Verfahren so umorganisiert, dass kürzere Teilfolgen immer zuerst bearbeitet werden.
- Ein Sortierverfahren mit dem gleichen asymptotischen Aufwand und ohne zusätzlichen Speicherbedarf ist Heapsort.

Open Distributed Sytems
ODS

Komplexität von vergleichenden Sortierverfahren

Komplexität von vergleichenden Sortierverfahren

- Alle bisher besprochenen Sortieralgorithmen gehören zur Klasse der Vergleichssortierverfahren
  - Elementaroperation ist der paarweise Größenvergleich

- Damit ergibt sich folgender Satz:

  Jedes Sortierverfahren, das auf dem paarweisen Vergleich von Elementen beruht, hat eine Komplexität von $\Omega(n \log n)$.

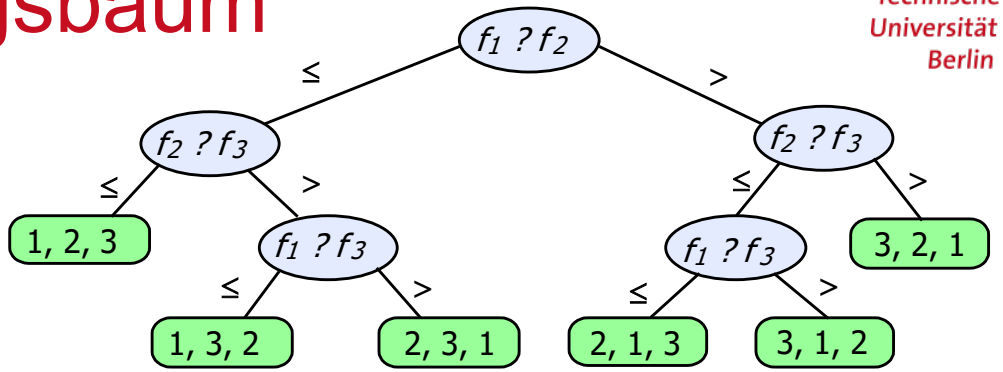
Komplexität von vergleichenden Sortierverfahren

- Alle bisher besprochenen Sortieralgorithmen gehören zur Klasse der Vergleichssortierverfahren
  - Elementaroperation ist der paarweise Größenvergleich

- Der Ablauf kann in einem binären Entscheidungsbaum wie folgt dargestellt werden:
  - Jeder interne Knoten entspricht dem Vergleich zweier Elemente.
  - Die jeweiligen Teilbäume enthalten die noch erforderlichen restlichen Vergleiche, so dass eine Sortierung gelingt.
  - Jedes Blatt repräsentiert eine der n! möglichen Permutationen.
  - Der Ablauf einer konkreten Sortierung entspricht einem Pfad von der Wurzel bis zu einem Blatt.

Entscheidungsbaum

- Beispiel für n=3



- Der längste Pfad von der Wurzel bis zu einem Blatt (= Höhe des Baumes) kennzeichnet die größtmögliche Anzahl von Vergleichen und damit die Laufzeit des Algorithmus.
  - Die Worst-case-Komplexität aller solcher vergleichsbasierter Sortierverfahren entspricht daher der Höhe des Baumes.
  - Eine untere Schranke für die Höhe des Entscheidungsbaums ist daher auch eine untere Schranke für jedes vergleichsbasierte Sortierverfahren.

Höhe des Entscheidungsbaums

- Wie groß ist die Höhe eines Entscheidungsbaumes für *n* Elemente mindestens?
- Es gibt *n!* Permutationen, also mindestens *n!* Blätter.
- Ein Binärbaum der Höhe *h* hat höchstens 2^h Blätter, daher gilt $n! \leq 2^h$
- Durch logarithmieren, $h \geq \log(n!)$
- Nach der Stirlingschen Formel gilt $n! > \left(\frac{n}{e}\right)^n$
- Also $h \geq \log\left(\left(\frac{n}{e}\right)^n\right) = n\log n - n\log e = O(n\log n)$

Vergleichende Sortierverfahren

Satz: Jedes Sortierverfahren, das auf dem paarweisen Vergleich von Elementen beruht, hat eine Komplexität von: $\Omega(n \log n)$.

Überblick

- Einfache Sortierverfahren
  - Sortieren durch Auswählen (selection sort)
  - Sortieren durch Vertauschen (bubble sort)
  - Sortieren durch Einfügen (insertion sort)
- Fortgeschrittene Sortierverfahren
  - Sortieren durch Gruppieren (quick sort)
  - Sortieren durch Mischen (merge sort)
- Nicht vergleichende Sortierverfahren
  - Sortieren durch Zählen (count sort)
  - Sortieren durch Fachverteilen (radix sort)

Nicht vergleichende Sortierverfahren

Schnelle, digitale Sortierverfahren

- Die bisher diskutierten Sortierverfahren beruhen auf dem paarweisen Vergleich der Werte der einzelnen Elemente.

- Eine Verbesserung der asymptotischen Komplexität dieser Verfahren ist nicht möglich, da sie eine untere Schranke von $\Omega(n \log n)$ haben.

Schnelle, digitale Sortierverfahren

Technische Universität Berlin

- Unter gewissen Einschränkungen des Wertebereichs können die Werte dazu verwendet werden, den endgültigen Platz direkt anzusteuern.
  - Sortieren durch Zählen (count sort)
  - Sortieren durch Fachverteilen (radix sort)
- Diese Verfahren sind jedoch nicht immer sinnvoll einsetzbar, z.B. wenn
  - das Sortieren stabil sein soll
  - der Wertebereich zu groß ist

Open Distributed Sytems
ODS

Wiederholung: Sortieren durch Zählen (count sort)

Technische Universität Berlin

- Annahme:
  - Die Werte stammen aus einem kleinen Wertebereich, d.h. sie liegen so dicht, dass sie zum Indizieren eines Arrays verwendet werden können.
  - Es ist wahrscheinlich, dass Werte mehrfach auftreten.
- Idee:
  - Die Häufigkeit jedes Elements wird ermittelt und daraus wird die endgültige Lage im Zielarray berechnet (streuendes Umspeichern).
  - Zum Schluss kann die Folge in das ursprüngliche Array zurückkopiert werden.

Open Distributed Sytems ODS

Beispiel: count sort

j:	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
A_in:	3	1	3	3	5	7	2	8	9	7	6	3	8	1	8	8

j:	1	2	3	4	5	6	7	8	9	10	
C:	2	1	4	0	1	1	2	4	1	0	Werte
	1	3	4	8	8	9	10	12	16	17	Zähler
											Position

i

A_out:	1	1	2	3	3	3	3	5	6	7	7	8	8	8	8	9
k:	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16

Komplexität: count sort

- Anzahl der Schritte
  - Linear in der Zahl der Elemente n
  - Linear in der Zahl der Werte N
- Also O(n), wenn N = O(n)
- Problem
  - In vielen praktischen Fällen ist der Wertebereich viel größer als die Zahl der Elemente

Sortieren durch Fachverteilen (radix sort)

- Idee:
  - Wir wenden *count sort* nacheinander auf die Stellen der Werte an
  - Begonnen wird bei der kleinsten signifikanten Stelle
  - Da *count sort* stabil ist, bleibt die Ordnung bezüglich der weniger signifikanter Stellen erhalten
- Voraussetzungen:
  - Werte können in einem Alphabet dargestellt werden
  - Die Buchstaben des Alphabets sind total geordnet
  - Typische Alphabete: Binär, Dezimal, Hexadezimal

Sortieren durch Fachverteilen (radix sort)

- Beispielfolge (Dezimal): 333, 78, 77, 3, 37, 38
- Sortieren nach der Einerstelle: 333, 3, 77, 37, 78, 38
  - 2 x 3 (333, 3), 2 x 7 (77, 37), 2 x 8 (78, 38)
- Sortieren nach der Zehnerstelle: 3, 333, 37, 38, 77, 78
  - 1 x 0 (3), 3 x 3 (333, 37, 38), 2 x 7 (77, 78)
- Sortieren nach den Hunderterstelle: 3, 37, 38, 77, 78, 333
  - 5 x 0 (3, 37, 38, 77, 78), 1 x 3 (333)
- Notwendige Größe für das Zählerfeld ist immer 10

Sortieren durch Fachverteilen (radix sort)

- Komplexität
  - Annahmen:
    - Der größte Wert ist N
    - Die Größe des Wertebereichs für eine Stelle ist b
  - Laufzeit:
    - Dann ist die Anzahl der Stellen proportional zu $\log_b N$
    - Für jede Stelle wird *count sort* verwendet, d.h. der Aufwand pro Stelle ist O(n)
    - Also insgesamt O(n log N), mit typischerweise log N << n

Rückblick

Technische Universität Berlin

VL 0 „Organisation und Inhalt“: Ablauf der Vorlesung, Termine
VL 1 „Algorithmen, Pseudocode, Sortieren I“: Insertion Sort
VL 2 „Algorithmen, Pseudocode, Sortieren II“: Selection Sort, Bubble Sort, Count Sort
VL 3 „Laufzeit und Speicherplatz“: Laufzeitanalyse der vorgestellten Sortierverfahren
VL 4 „Einfache Datenstrukturen“: Arrays, verkettete Listen, Structs in C, Stack, Queue
VL 5 „Bäume“: Binärbäume, Baumtraversierung, Laufzeitanalyse Baumoperationen
VL 6 „Teile und Herrsche I“: Einführung der algorithmischen Methode, Merge Sort
VL 7 „Korrektheitsbeweise“: Rechnermodel, Beispielbeweise
VL 8 „Dateien in C“: Dateien, Dateisysteme, Verzeichnisse, Dateiverwaltung mit C
VL 9 „Prioritätenschlangen/Halden/Heaps “: Heap Sort, Binärer Heap, Heap Operationen
VL 10 „Fortgeschrittene Sortierverfahren“: Quick Sort, Radix Sort
VL 11 „AVL Bäume“: Definition, Baumoperationen, Traversierung
VL 12 „Teile und Herrsche II“: Generalisierung des algorithmischen Prinzips, Mastertheorem
VL 13 „Q & A“: Offene Vorlesung/Wiederholung

Open Distributed Sytems
ODS